

Hide-and-Seek Drone Swarm

CentraleSupélec Multi-Agent Systems — Technical Report

May 28, 2026

Abstract

This report describes the design and implementation of a multi-agent hide-and-seek system in which a single airborne *seeker* drone tries to spot a group of *hider* drones that hide behind obstacles. The system is built on top of the CentraleSupélec Voliere TP framework: every drone controller is a Python function that the simulator (`standalone_sim.py`) calls each tick with the current world state, and must return a velocity command. The contribution lives in the modules around `tp_algos.py`: (i) an obstacle-consensus law with both an asymptotic linear term and a barrier term, used by hiders and seeker alike; (ii) a Coulomb-form inter-agent dispersion consensus on the swarm potential $V = (k/2) \sum_{i \neq j} 1/\|p_i - p_j\|$ that the hiders descend collectively; (iii) a forward field-of-view sensor with a single `hider_view_heading` function that drives *both* the catch predicate and the rendered cone, so a hider can never be seen if its on-screen cone does not also see the seeker; (iv) a shared-memory broadcast channel between hiders; (v) a *cover-assignment* primitive that solves a deterministic min-cost matching so each hider takes a distinct cover face relative to the last-known seeker pose, with a 1 s / 0.5 m hysteresis cache that prevents role-swap chatter; and (vi) a controller-level 3D inter-drone repulsion in `tp_algos.py` that adds a hard “never closer than 1 m” fence on top of the per-controller behaviours. The hider mixes a moderate-gain Lissajous roam (with analytic velocity feed-forward), the obstacle consensus, the dispersion consensus, the 3D drone fence, and a soft boundary repulsion; when an alert is fresh the goal target switches to the assigned cover face and the gain is raised so the hider actually *reaches* the face before the seeker. The seeker uses the same obstacle consensus plus a stronger *soft boundary repulsion* of its own (added because the seeker has no dispersion term pulling it inward), and lands once the referee reports all hiders caught.

Contents

1	Project context and high-level architecture	3
2	Mathematical model	6
2.1	State and kinematics	6
2.2	Obstacles	6
3	Hider controller (<code>cf2_milestone1.py</code>)	6
3.1	Force composition	6
3.2	Lissajous roaming path	7
3.3	Mode switching: roam vs. hide	8
3.4	Cover-assignment geometry (multi-agent)	9
4	Consensus laws (<code>cf2_consensus.py</code>)	10
4.1	Obstacle consensus	13
4.2	Inter-agent dispersion consensus	13

5 Sensing (cf2_sensing.py)	17
5.1 Intent heading	17
5.2 FoV-cone obstacle sensing	17
6 Seeker controller (rmtt_seeker.py)	17
6.1 Sweep path	17
6.2 Control law	18
6.3 Soft boundary repulsion	18
6.4 Search cone and line-of-sight	19
7 Inter-hider communication (cf2_hider.py)	19
8 Catch detection (game_referee.py)	21
9 Visualization (sim_viz.py)	22
10 Simulator hooks and controller-level separation (tp_algos.py)	22
10.1 3D inter-drone barrier: a hard 1 m floor	22
10.2 How the helper is used	23
10.3 Hider takeoff handshake	23
11 Simulation infrastructure (standalone_sim.py)	23
11.1 World and hardware specs	23
11.2 State machine	24
11.3 Async controller execution	24
11.4 Collision detection	24
12 Tunables summary	24
13 Discussion and design choices	26
14 Future work	27
A File listing	28

1 Project context and high-level architecture

The project is a CentraleSupélec TP (2A/3A) on multi-agent systems. The target hardware platforms are Crazyflie 2 nano-quadrotors (acting as hiders) and a DJI RoboMaster TT drone (acting as the seeker), flying inside the school’s *Volière* arena. The pedagogical aim is to use a hide-and-seek game as an accessible illustration of multi-agent control, both for the students implementing it and for the children visiting the Volière.

Rules of the game.

- A single seeker drone (RMTT) cruises the arena on a slow sweep path. It *never receives the hiders’ positions* – it has to find them with vision.
- Several hider drones (CF2) roam the arena. They know the seeker’s position only through *their own vision* (a forward cone), and only communicate sightings to each other through a shared broadcast channel.
- When the seeker’s forward cone contains a hider *and* the line of sight is not occluded by an obstacle, the hider is “caught” and triggered to land.

Module decomposition. The code base is split into small files that each address one concern:

Module	Role
<code>standalone_sim.py</code>	The simulator. Owns the world state, calls each robot’s controller in a thread pool, integrates kinematics, detects collisions, renders the 3D figure. The only edit in this work is the spawn-list block (4 hiders, 3 obstacles); everything else is upstream.
<code>tp_algos.py</code>	The required entry-point file (prof requirement). Beyond the dispatch shims, it hosts a controller-level 3D inter-drone repulsion (<code>_compute_drone_repulsion_3d</code>) and applies it on top of both the seeker and hider commands. <i>Edits kept inside the marked TO BE MODIFIED blocks.</i>
<code>cf2_milestone1.py</code>	Top-level hider controller: composes a goal force (roam or hide, with analytic Lissajous feed-forward in roam), obstacle consensus, Coulomb dispersion consensus, and boundary repulsion; switches modes on the shared seeker-sighting channel. Also exports <code>hider_view_heading</code> , the single source of truth for the FoV axis used by both the catch predicate and the rendered cone.
<code>cf2_sensing.py</code>	The hider’s sensing primitives: intent heading and FoV-cone obstacle sensing on an axis-aligned bounding box (AABB) model.
<code>cf2_consensus.py</code>	Two consensus laws: the obstacle push (linear + barrier, used by both hiders and seeker) and the inter-agent dispersion law (Coulomb-form gradient flow, used by the hiders to maximize pairwise distances).
<code>cf2_hider.py</code>	Inter-hider communication (shared seeker-sighting memory), single-target cover geometry, and the multi-agent cover-assignment solver (deterministic min-cost matching).

<code>rmtt_seeker.py</code>	Seeker controller (<code>compute_seeker_cmd</code> , which does <i>not</i> accept hider positions) and the <code>can_see</code> predicate shared with the referee and the hiders' seeker-detection logic.
<code>game_referee.py</code>	Per-frame catch detection using the same <code>can_see</code> predicate as the seeker's vision cone.
<code>sim_viz.py</code>	Visualization overlays: 3D FoV cones, position trails, status HUD.

Data flow. At each tick $t = k \Delta t$ ($\Delta t = 0.05$ s), the simulator:

1. Snapshots all pose arrays so the controllers see a consistent world.
2. Submits each robot's controller (the public `tp_algos._controller` entry points) to a thread pool. The controller returns a target velocity (and for drones, a target altitude / takeoff / land flags).
3. Integrates the returned commands with a simple Euler step, applies the actuator-rate saturations, adds a Gaussian position noise term ($\sigma = 5$ mm per step).
4. Calls `game_referee.update_catches` to mark any catches.
5. Runs collision detection (boundary, obstacle, robot-to-robot) and colours the robots' safety globes red on contact.
6. Calls `sim_viz.update` to refresh cones, trails and the HUD.

Default scenario. The simulator ships with `nbRMTT = 1` seeker, `nbCF2 = 4` hiders, and `nbObstacle = 3` (Fig. 1):

- A $1.0 \times 0.5 \times 2.5$ m pillar at $(-0.5, 1.5, 0)$.
- A $0.5 \times 1.1 \times 2.5$ m pillar at $(0.8, -2.0, 0)$.
- A $3.0 \times 1.0 \times 2.5$ m *central slab* at $(0, 0, 0)$ which splits the arena into a north room ($y > 0.5$) and a south room ($y < -0.5$) and reduces direct sight-lines between them.

The two side pillars occupy opposite rooms; together with the slab they expose, depending on the seeker pose, up to $3 \cdot 4 = 12$ axis-aligned candidate cover faces – the four cardinal normals of each box, kept only when the outward normal points away from the seeker, see Sec. 3.4. With four hiders this is overdetermined in most configurations and the cover-assignment law has slack to spread the swarm across both rooms. The seeker spawns at $(1.5, 1.0, 0)$, just north of the central slab and just east of the small north pillar; this is its worst-case spawn because the slab puts the south-room hiders out of line-of-sight immediately. The four CF2 hiders start at the corners of an inset rectangle, $(\pm 1.8, \pm 3.8, 0)$. The corner layout satisfies four simultaneous constraints at $t = 0$: pairwise hider spacing ≥ 3.6 m (well above the 1 m hard floor of Sec. 10), seeker-to-hider distance ≥ 2.82 m (outside the seeker's 2.0 m vision range, so no hider is in the seeker's cone at spawn – and the seeker takes off in state 1 for 3 s before catch detection is even active), wall clearance ≥ 0.7 m (outside the 0.6 m boundary repulsion margin), and obstacle clearance ≥ 1.05 m from every box (well above $r_{\text{safe}} = 0.6$ m). Each room of the arena starts with two hiders, so the cover-assignment law always has candidates on both sides regardless of where the seeker's sweep takes it first.

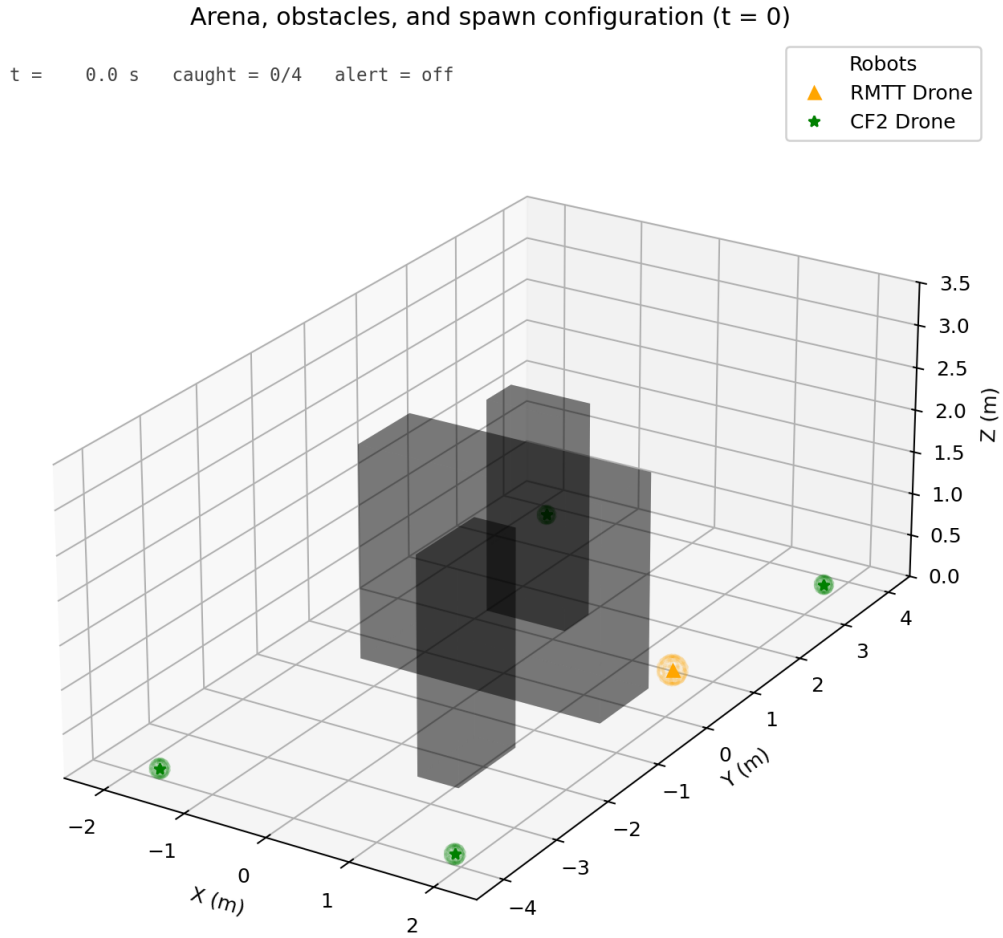


Figure 1: Default scenario at $t = 0$. Seeker (orange triangle) at $(1.5, 1.0, 0)$; four hidrs (green stars) at $(\pm 1.8, \pm 3.8, 0)$; three AABB obstacles in dark grey. The central slab is the dominant occluder along the y axis and is what makes the south-room hidrs invisible to the seeker at spawn.

2 Mathematical model

2.1 State and kinematics

Each drone is modelled as a point mass with state $\mathbf{p}_i = (x_i, y_i, z_i)^\top \in \mathbb{R}^3$ and a velocity command $\mathbf{u}_i = (v_x, v_y, v_z)^\top$. The integration is a forward Euler step with Gaussian noise:

$$\mathbf{p}_i(t + \Delta t) = \mathbf{p}_i(t) + \mathbf{u}_i \Delta t + \boldsymbol{\eta}_i, \quad \boldsymbol{\eta}_i \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_3), \quad \sigma = 0.005 \text{ m}.$$

Before integration the simulator applies a 3D magnitude clamp at the actuator limit (`clamp_vel3d` in `standalone_sim.py`).

For CF2 the controller actually returns $(v_x, v_y, z_{\text{dist}})$ where z_{dist} is the *desired altitude*. The simulator converts it to a vertical velocity $v_z = z_{\text{dist}} - z$ and applies the 3D clamp on (v_x, v_y, v_z) together.

2.2 Obstacles

Obstacles are axis-aligned bounding boxes (AABBs) standing on the floor:

$$\mathcal{B}_j = [o_x - \frac{s_x}{2}, o_x + \frac{s_x}{2}] \times [o_y - \frac{s_y}{2}, o_y + \frac{s_y}{2}] \times [0, s_z].$$

The closest point on $\mathcal{B}_j$ to a query point $\mathbf{p}$ is the per-axis clamp of $\mathbf{p}$ into the box, which collapses to $\mathbf{p}$ itself when $\mathbf{p} \in \mathcal{B}_j$.

3 Hider controller (`cf2_milestone1.py`)

3.1 Force composition

The controller assembles four terms:

$$\mathbf{F}_{\text{total}} = \mathbf{F}_{\text{goal}} + \mathbf{F}_{\text{obs}} + \mathbf{F}_{\text{disp}} + \mathbf{F}_{\text{bnd}},$$

and outputs the planar component of $\mathbf{F}_{\text{total}}$ after a magnitude saturation at $v_{\text{max}} = 0.55 \text{ m/s}$ (just below the hardware cap `MAX_V_CF2` = 0.6 m/s). The vertical channel is governed separately: the controller returns a desired altitude z_{dist} that the simulator converts to $v_z = z_{\text{dist}} - z$ and then clamps with (v_x, v_y, v_z) *together* at `MAX_V_CF2`. To stop a large v_z from eating the xy avoidance budget inside that 3D clamp, the controller applies a per-call rate limit `_Z_RATE_LIMIT` = 0.01 m on $|z_{\text{dist}} - z|$ before returning. The result: the xy channel keeps its full 0.55 m/s magnitude even during a vertical excursion, the obstacle barrier of Sec. 4 stays fully effective, and the drone climbs or descends at most 0.2 m/s.

Goal attraction (mode-dependent)

The goal target is the Lissajous waypoint $\mathbf{p}_{\text{lis}}$ in roam mode and the assigned cover face $\mathbf{c}_i$ in hide mode (Sec. 3.4). The two modes use *different* gain laws:

$$\begin{aligned} \mathbf{F}_{\text{goal}}^{\text{roam}} &= k_g^{\text{roam}} (\mathbf{p}_{\text{lis}} - \mathbf{p}) + \dot{\mathbf{p}}_{\text{lis}}, & k_g^{\text{roam}} &= 0.45, \\ \mathbf{F}_{\text{goal}}^{\text{hide}} &= k_g^{\text{hide}} (\mathbf{c}_i - \mathbf{p}), & k_g^{\text{hide}} &= 0.9. \end{aligned}$$

The analytic velocity feed-forward $\dot{\mathbf{p}}_{\text{lis}}$ is the exact time derivative of the Lissajous waypoint, returned by `_lissajous_velocity` in `cf2_milestone1.py:83-89`. With the feed-forward in place, k_g^{roam} only has to correct *tracking error*, not produce the entire chase velocity; this lets it stay moderate, so a small obstacle excursion or a dispersion push easily deflects the drone off-path. The cover target is stationary ($\dot{\mathbf{c}}_i = 0$), so the feed-forward is dropped. The hide gain is the highest in the controller because the drone must *actually reach* the assigned face inside the seeker-memory TTL: at $0.9 \times 0.6 \text{ m} \approx v_{\text{max}}$ the planar saturation engages whenever the hider is more than $\sim 0.6 \text{ m}$ from cover, so the hide command is essentially “go full speed straight to the face”.

Obstacle consensus

Each sensed obstacle j contributes (see Sec. 4)

$$\mathbf{F}_{\text{obs}}^{(j)} = \underbrace{k_{\text{obs}} \max(0, r_{\text{safe}} - d_j)}_{\text{linear}} \mathbf{n}_j + \underbrace{k_{\text{bar}} \left(\frac{r_{\text{safe}}}{\max(d_j, \varepsilon)} - 1 \right)}_{\text{barrier}} \mathbf{n}_j,$$

with d_j the distance to the obstacle surface, $\mathbf{n}_j$ the outward normal (surface $\rightarrow$ drone), and $\varepsilon = 0.02 r_{\text{safe}}$.

Inter-agent dispersion consensus

Per-agent gradient of the swarm potential $V_{\text{disp}}(\mathbf{p}_1, \dots, \mathbf{p}_N) = \frac{k_{\text{disp}}}{2} \sum_{i \neq j} 1/\|\mathbf{p}_i - \mathbf{p}_j\|$ in the xy plane:

$$\mathbf{F}_{\text{disp}}^{(i)} = -\nabla_{\mathbf{p}_i} V_{\text{disp}} = k_{\text{disp}} \sum_{j \neq i} \frac{\mathbf{p}_i - \mathbf{p}_j}{\|\mathbf{p}_i - \mathbf{p}_j\|^3}, \quad k_{\text{disp}} = 0.4.$$

This is a true multi-agent consensus law on the aggregate cost V_{disp} : every hider’s local law decreases the *same* scalar function along the joint gradient flow, so the swarm collectively minimizes V_{disp} , equivalently maximizes inverse pairwise distances, equivalently spreads as far apart as the boundary and obstacle constraints allow. Unlike a short-range repulsion, the law acts at *all* distances – every hider always knows about every other hider’s pull. Symmetry implies the centroid of the swarm is invariant under $\mathbf{F}_{\text{disp}}$ alone (the goal, obstacle, and boundary terms then steer the centroid). A numerical floor $\varepsilon_{\text{disp}} = 0.05$ m on the denominator avoids a blow-up if two hidrs briefly co-locate. A stricter 3D “never closer than 1 m” separation between *all* flying drones (hidrs *and* seeker) is added on top in `tp_algos.py` (Sec. 10).

Boundary repulsion

Each wall at distance d inside a margin $m = 0.6$ m pushes inward:

$$F_{\text{bnd}}^{(w)} = g_{\text{bnd}} \left(\frac{1}{\max(d, 10^{-4})} - \frac{1}{m} \right), \quad g_{\text{bnd}} = 0.09.$$

The gain is intentionally small (0.09) – the goal is a gentle nudge, not a hard bounce.

3.2 Lissajous roaming path

Each hider i tracks a 3D Lissajous waypoint

$$\mathbf{p}_{\text{lis}}^{(i)}(t') = \begin{pmatrix} c_x + A_x \sin(\omega_x^{(i)} t' + \varphi_x^{(i)}) \\ c_y + A_y \sin(\omega_y^{(i)} t' + \varphi_y^{(i)}) \\ c_z \end{pmatrix},$$

with $A_x = 1.6$, $A_y = 3.0$, $c_z = 1.0$, and per-axis, per-drone angular frequencies

$$\omega_x^{(i)} = 0.16 + 0.018(i - 1), \quad \omega_y^{(i)} = 0.11 + 0.015(i - 1) \text{ rad/s},$$

and coprime phase offsets $\varphi_x^{(i)} = (i - 1) \cdot 2\pi/3$, $\varphi_y^{(i)} = (i - 1) \cdot 2\pi/5$. The two axes use different base frequencies so a single drone alone already traces a non-degenerate Lissajous, and the small per-drone increments stop the swarm from collapsing onto a shared orbit (Fig. 7).

Per-drone phase offset (no spawn lurch). The time argument is $t' = t + \tau_i$ where τ_i is set lazily on the first call to the controller: a coarse 0–60s search picks the τ_i for which $\mathbf{p}_{\text{lis}}^{(i)}(\tau_i)$ is closest to the drone’s spawn pose (`cf2_milestone1._ensure_t_offset`, lines 97–114). Without this, the drone would face a ~ 3 m initial error at $t = 0$ and the analytic feed-forward $\dot{\mathbf{p}}_{\text{lis}}$ would have nothing useful to feed forward into: the result was a violent spawn lurch followed by a corner-cutting transient that often clipped an obstacle.

Z scaffolding present but disabled. The code base carries a complete z branch – amplitude $A_z = 0.5$, frequency $\omega_z^{(i)} = 0.09 + 0.013(i - 1)$, phase $\varphi_z^{(i)} = (i - 1) \cdot 2\pi/7$, and a z -component in the FoV axis – but every z -coupled line is currently commented out. Specifically:

- `cf2_milestone1._lissajous_target` returns $t_z = c_z$ (the $A_z \sin(\omega_z t)$ term is commented).
- `cf2_milestone1._lissajous_freqs_phases` overrides $\omega_z = 0$ (line 67).
- `cf2_sensing.intent_heading` sets $dz = 0$ (line 38), making the unit-vector cone axis a 2D heading.
- `cf2_sensing.sense_obstacles` sets $dz = 0$ in the closest-point delta (line 88), so the cone test is purely in the xy plane.

The state was introduced by the commits `0f19e8f / 7fcbcc0` (“*modif de vitesses z*”). The reason is the actuator clamp: the simulator clamps the 3D vector (v_x, v_y, v_z) at `MAX_V_CF2` together, and the per-step position noise on z ($\sigma = 5$ mm) together with the 3D drone fence in z would on bad ticks force v_z up near `MAX_V`, eating the xy avoidance budget and breaking the discrete-time non-penetration argument of Sec. 10. Re-enabling Z is a one-line uncomment in each location once v_z is given its own private saturation; this is in the Future Work list.

3.3 Mode switching: roam vs. hide

The hidere communicate seeker sightings through a single module-level dict in `cf2_hider.py`. The mode logic in `compute_roaming_cmd` is:

1. Compute the FoV-based seeker detection. The hider’s forward cone has half-angle $\theta_{\text{fov}}^{\text{hider}} = 45^\circ$ (down from the usual 60° : a narrower cone gives fewer false alerts on a seeker that is nearby but not actually aimed at the hider, which would otherwise lock the swarm into hide mode for the full 5s TTL even when the threat has moved on) and range 3.0m. If `can_see(self, seeker, hider_view_heading,  $\theta_{\text{fov}}^{\text{hider}}$ , 3.0m)`, call `report_seeker_sighting(seeker_pos, t)`.
2. Read the shared memory via `get_last_seen(t)`. If a sighting is fresher than TTL = 5s, switch the goal target to the assigned cover point (next section); otherwise, use the Lissajous target. Roam-mode visualization is shown in Fig. 2.

The structural asymmetry is important: the seeker controller never reads or writes the shared memory, so “hidere can talk to each other but the seeker can’t listen” is enforced by interface, not by convention.

One heading function for both vision and rendering. The FoV axis is computed by `hider_view_heading` in `cf2_milestone1.py:143-148`. That function calls `intent_heading` after composing the per-drone phase offset τ_i described above, so the cone axis always corresponds to the direction the drone is actually trying to go. Both `can_see` (called from inside `compute_roaming_cmd`) and the rendered cone in `sim_viz.update` read this exact function. The architectural consequence is that an alert is impossible without a corresponding on-screen cone hit – the visualization cannot lie about what the controller saw. This was a deliberate refactor

(commit 45685af); previously the on-screen cone was rebuilt from a duplicated copy of the Lissajous formulas in `sim_viz` that did not carry τ_i , so the rendered cone and the actual sighting cone drifted apart over the first few seconds of every run.

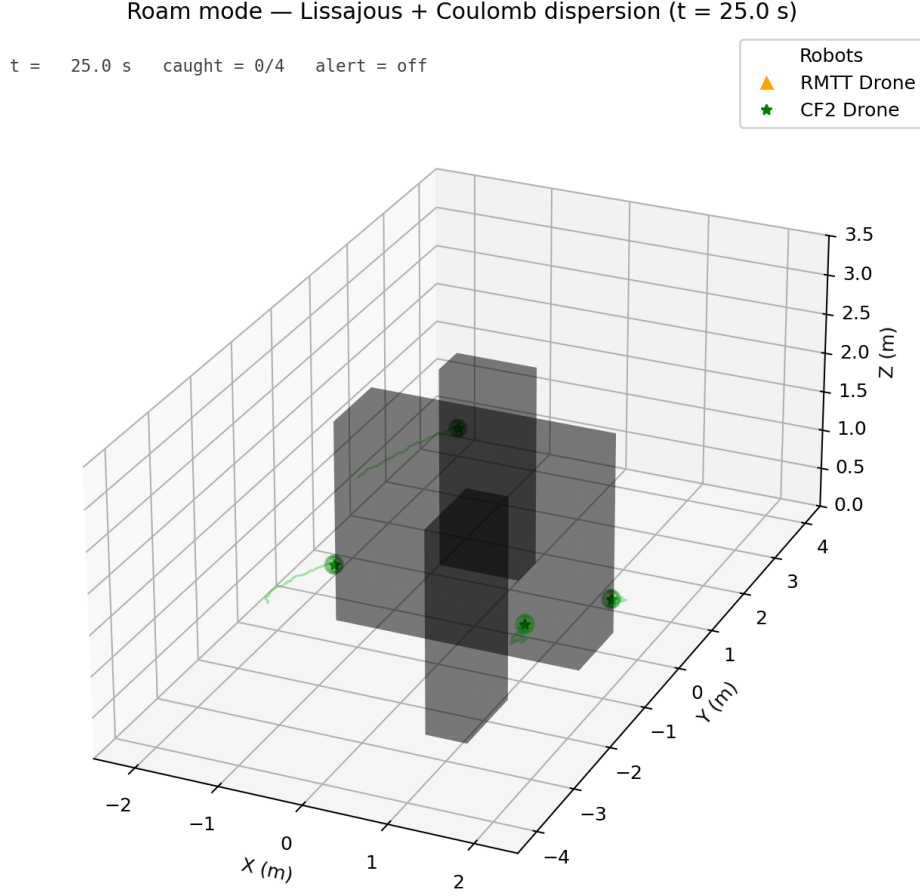


Figure 2: Roam mode at $t = 25$ s. With the seeker parked out of vision range, the swarm sits in its Coulomb-dispersion equilibrium inside the arena: hidere are pushed apart by $-\nabla V_{\text{disp}}$, the boundary repulsion stops them from sticking to the walls, and the analytic Lissajous feed-forward keeps every hider drifting along its own quasi-orbit rather than freezing at a fixed point. Cones omitted to declutter the swarm geometry.

3.4 Cover-assignment geometry (multi-agent)

Hide mode is a small *cooperative task-allocation* problem: there are several cover spots and several hidere, and the swarm wants every hider on a distinct spot so the seeker cannot harvest them in one cone sweep. The solution lives in `cf2_hider.assign_cover_targets` and runs in four steps; the result is visualised in Fig. 3 and Fig. 4.

1. Candidate enumeration. For every obstacle (center $\mathbf{o}_j$, half-extents (h_x, h_y)) and every AABB face with outward normal $\hat{\mathbf{n}} \in \{\pm\hat{\mathbf{e}}_x, \pm\hat{\mathbf{e}}_y\}$, build the candidate

$$\mathbf{c}_{j,\hat{\mathbf{n}}} = \underbrace{\mathbf{o}_j + (\hat{\mathbf{n}} \cdot \mathbf{h}) \hat{\mathbf{n}}}_{\text{face center}} + d_{\text{cov}} \hat{\mathbf{n}}, \quad d_{\text{cov}} = 0.8 \text{ m},$$

where $\mathbf{h} = (h_x, h_y)$. The face is *kept* only if $\hat{\mathbf{n}} \cdot (\mathbf{o}_j - \mathbf{s}) > 0$, i.e. the normal points away from the seeker – only those faces actually occlude line of sight to the cover point. With the three obstacles of the default scenario, $3 \times 4 = 12$ candidates are generated before filtering; in a typical seeker pose roughly half survive the directional gate. The $d_{\text{cov}} > r_{\text{safe}}$ inequality is deliberate: the candidate sits *outside* the obstacle-consensus push zone, so a hider can come to rest on it without fighting the avoidance law.

2. Cost matrix. With N hidiers and $M \geq N$ candidates, define $C_{ij} = \|\mathbf{p}_i^{xy} - \mathbf{c}_j\|_2^2$. Squared distance gives the same min-cost solution as Euclidean distance and avoids $N \cdot M$ square roots.

3. Min-cost injective assignment. Find $\pi : \{1, \dots, N\} \rightarrow \{1, \dots, M\}$ injective minimizing $\sum_i C_{i, \pi(i)}$. For the default scenario $(N, M) \leq (4, 12)$ the search space is $P(M, N) \leq 11,880$ permutations, so a branch-and-bound brute force is faster and dependency-free compared to importing SciPy’s `linear_sum_assignment`. The implementation prunes any partial assignment whose accumulated cost already exceeds the best complete one (`cf2_hider._best_assignment`).

4. Hysteresis cache. The matching is *not* recomputed every tick. A small cache (`_cover_assignment_cache`) stores the last solved assignment and re-uses it as long as both of these hold:

- `age = t - t_{\text{cache}} < _COVER_CACHE_MAX_AGE = 1.0\text{s}`, and
- the seeker has moved less than `_COVER_CACHE_SEEKER_DELTA = 0.5\text{m}` since the cache was built.

Without this gate the cost matrix would shift slightly every tick on position noise alone, occasionally swapping two hidiers’ assigned faces when their costs cross – a role swap that does not change the steady-state formation but does send the affected pair on a wasted round-trip in the meantime. The cache also drops the effective matching frequency from 20 Hz to ~ 1 Hz, which would matter on real flight hardware.

Decentralization. The simulator hands every hider the same `cf2_poses` matrix and the same shared seeker sighting; `assign_cover_targets` is deterministic in those inputs and so is the cache. Every hider independently computes the *same* matching and reads its own row from the result – a consensus reached by “every node solves the same optimization on the same inputs” rather than by message exchange. No explicit communication is needed beyond the existing shared-memory sighting channel.

Fallback. If $M < N$ (the seeker hides most far-side faces), the $N - M$ leftover hidiers fan out radially around the seeker at distance 1.5 m on angles $2\pi k / (N - M)$. They no longer get a true occlusion but they no longer pile up either, and the dispersion consensus keeps pushing them apart in the meantime. The single-target `seek_cover_target` function (older nearest-cover-only behaviour) is kept exported for backward compatibility and tests, but is no longer on the hot path.

4 Consensus laws (`cf2_consensus.py`)

The module hosts two consensus laws used by the hidiers. The obstacle consensus (Sec. 4.1) is also used by the seeker. The inter-agent dispersion law (Sec. 4.2) is hider-only and is what gives the swarm its coordinated “spread out” behaviour.

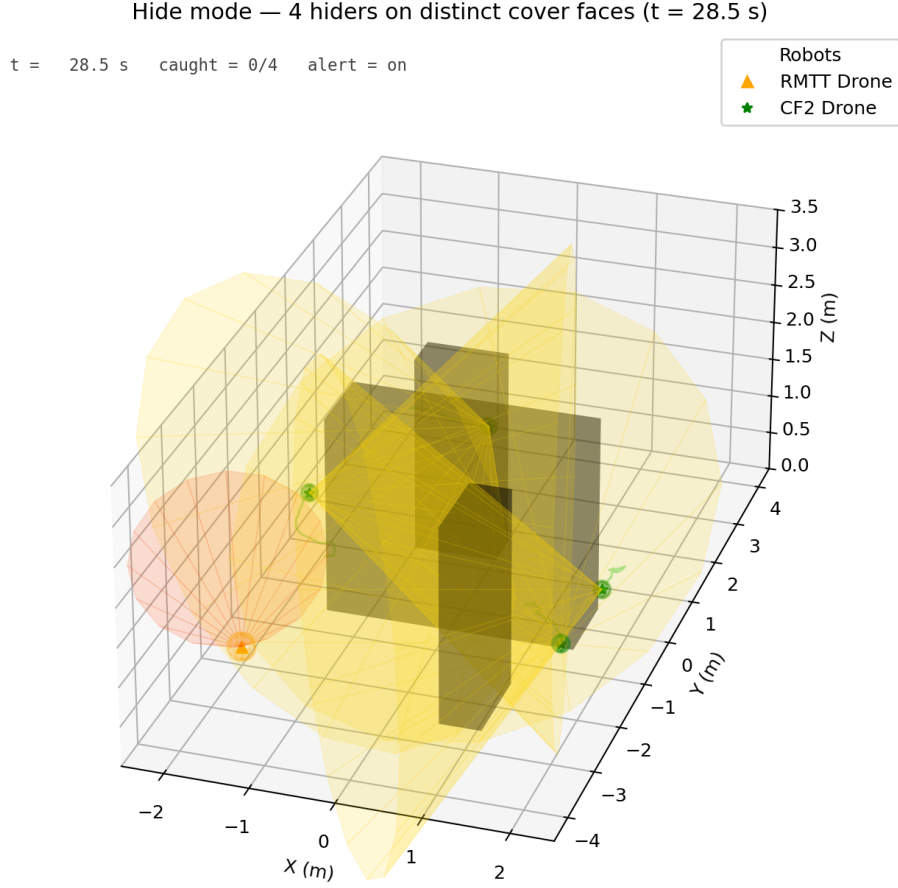


Figure 3: Hide mode (oblique 3D). The seeker has just emitted a sighting from $(0, -4, 0.8)$ (south wall, ahead of obstacle 2). The four hidrs' cones switch to gold (alert), each hider has been assigned a distinct far-side cover face, and the swarm converges within ~ 3 s (well inside the 5 s TTL).

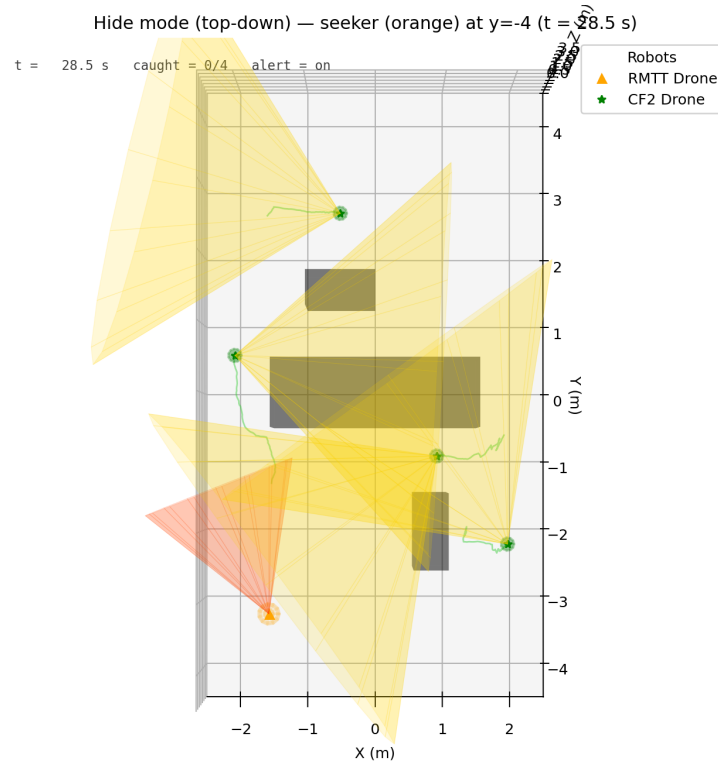


Figure 4: Same hide-mode instant viewed from straight above. Top-down makes the cover-face geometry unambiguous: each hider sits behind one obstacle face whose outward normal points *away* from the seeker's south-side sighting, and no two hiders share a face – the visual signature of the injective min-cost matching.

4.1 Obstacle consensus

Definition

Given a list of sensed obstacles, each with closest-surface distance d_j and outward normal $\mathbf{n}_j$, the consensus command is

$$\mathbf{u} = \sum_{j: d_j < r_{\text{safe}}} \left[k_{\text{obs}} (r_{\text{safe}} - d_j) + k_{\text{bar}} \left(\frac{r_{\text{safe}}}{\max(d_j, \varepsilon)} - 1 \right) \right] \mathbf{n}_j.$$

The function does *not* saturate; saturation is the caller’s responsibility (the hider controller applies the magnitude cap on the summed force, after adding the goal/separation/boundary contributions).

Properties

Asymptotic (linear term). Define $e_j(\mathbf{p}) = \max(0, r_{\text{safe}} - d_j(\mathbf{p}))$ and the Lyapunov candidate

$$V(\mathbf{p}) = \frac{1}{2} \sum_j e_j(\mathbf{p})^2.$$

For static obstacles, $\nabla d_j = \mathbf{n}_j$ on the violating set, so the purely-linear closed loop $\dot{\mathbf{p}} = -k_{\text{obs}} \sum_j e_j \mathbf{n}_j$ satisfies $\dot{V} = -k_{\text{obs}} \sum_j e_j^2 \leq 0$, with equality only when all violations vanish. The drone therefore converges to the safe set $\{\mathbf{p} : d_j(\mathbf{p}) \geq r_{\text{safe}} \forall j\}$ – an asymptotic consensus on “be outside every safety bubble”.

Bounded actuation (barrier term). The linear term alone has bounded magnitude on bounded violations, so a strong enough goal-attraction force could overpower it close to the obstacle. The barrier term grows like r_{safe}/d_j as $d_j \rightarrow 0$, which *always* dominates any bounded attraction once the drone is close enough. Combined with the actuator saturation in the caller, the closed-loop velocity is bounded everywhere while the dynamics inside $\{d_j < r_{\text{safe}}\}$ remain dominated by the avoidance push.

Numerical safety. The denominator uses $\max(d_j, \varepsilon)$ with $\varepsilon = 0.02 r_{\text{safe}} = 12 \text{ mm}$. In simulation a drone may briefly penetrate an obstacle due to the discrete integration step; this clamp keeps the law finite while still producing a very large push. Fig. 5 (left) shows the linear and barrier components plotted side by side: the linear term carries all of the restoring force at moderate violations while the barrier dominates as $d \rightarrow 0$, exactly the property the controller relies on to remain bounded under any bounded goal attraction.

4.2 Inter-agent dispersion consensus

Definition

For the hider swarm $\{\mathbf{p}_1, \dots, \mathbf{p}_N\} \subset \mathbb{R}^2$ (only the planar component enters the law), define the aggregate cost

$$V_{\text{disp}}(\mathbf{p}_1, \dots, \mathbf{p}_N) = \frac{k_{\text{disp}}}{2} \sum_{\substack{i,j \\ i \neq j}} \frac{1}{\|\mathbf{p}_i - \mathbf{p}_j\|}.$$

Each hider runs the per-agent gradient descent

$$\mathbf{u}_i = -\nabla_{\mathbf{p}_i} V_{\text{disp}} = k_{\text{disp}} \sum_{j \neq i} \frac{\mathbf{p}_i - \mathbf{p}_j}{\|\mathbf{p}_i - \mathbf{p}_j\|^3}.$$

This is the Coulomb form: agent i feels an inverse-square push (in *magnitude* per unit charge) from every other agent. The implementation floors $\|\mathbf{p}_i - \mathbf{p}_j\|$ at $\varepsilon_{\text{disp}} = 0.05 \text{ m}$ to keep the gradient bounded if two hidens briefly co-locate.

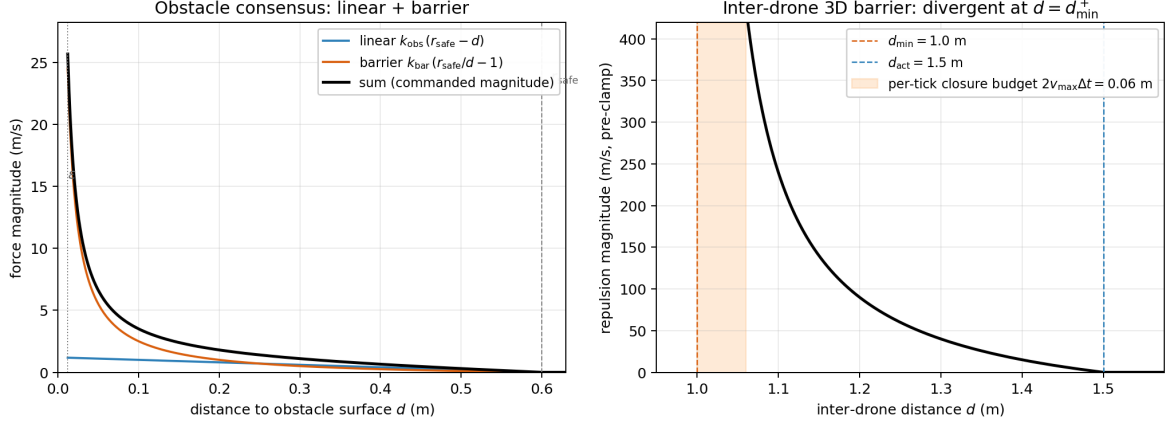


Figure 5: Left: obstacle consensus magnitude vs. distance to the obstacle surface, decomposed into the linear term $k_{\text{obs}}(r_{\text{safe}} - d)$ and the barrier term $k_{\text{bar}}(r_{\text{safe}}/d - 1)$. The numerical clamp at $\varepsilon = 0.02 r_{\text{safe}}$ caps the barrier at ~ 25 m/s pre-saturation. Right: inter-drone 3D barrier magnitude vs. pairwise distance over the active band $[d_{\min}, d_{\text{act}}]$. The shaded strip is the per-tick closure budget $2v_{\max}\Delta t = 0.06$ m – the barrier is already in the hundreds of m/s by the time a relative-closure step could cross it, which underwrites the discrete-time invariance argument of Sec. 10.

Properties

Aggregate Lyapunov. V_{disp} is itself a Lyapunov function for the joint flow $\dot{\mathbf{p}}_i = \mathbf{u}_i$ on the unconstrained domain $\{\mathbf{p}_i \neq \mathbf{p}_j \forall i \neq j\}$:

$$\dot{V}_{\text{disp}} = \sum_i \nabla_{\mathbf{p}_i} V_{\text{disp}} \cdot \dot{\mathbf{p}}_i = - \sum_i \|\nabla_{\mathbf{p}_i} V_{\text{disp}}\|^2 \leq 0,$$

with equality only at critical configurations of V_{disp} . Every hider’s local descent decreases the *same* swarm-level scalar – the defining property of a consensus law on an aggregate cost.

Long-range coupling. There is no cutoff: every other hider always contributes. Contrast with a short-range repulsion (e.g. $1/d^2$ inside a 0.9 m influence radius and zero outside): such a law is silent the moment the swarm is more than 0.9 m apart, so it cannot push two well-separated hidens further apart. The Coulomb form pushes at any distance, so the equilibrium is determined by the boundary and obstacle constraints, not by the law going silent.

Centroid invariance and bounded equilibria. $\mathbf{F}_{\text{disp}}^{(j \rightarrow i)} = -\mathbf{F}_{\text{disp}}^{(i \rightarrow j)}$ (Newton’s third law in form), so the swarm centroid is invariant under $\mathbf{F}_{\text{disp}}$ alone. Bounded equilibria emerge from the combination with the boundary repulsion of Sec. 3.1: the walls confine the swarm to the arena while $\mathbf{F}_{\text{disp}}$ pushes the hidens to the configuration with maximum pairwise distances inside that confinement.

Decentralization. The law uses only $\mathbf{p}_i$ and $\{\mathbf{p}_j\}_{j \neq i}$ – the simulator-provided pose matrix is the only neighbourhood query needed, no extra messaging. In a real deployment the assumed information flow is “each hider broadcasts its position to all the others at controller frequency”. The shared-memory channel already exists for seeker sightings (Sec. 3.4) and would extend trivially to position broadcasts.

Dispersion field $-\nabla_{\mathbf{p}_i} V_{\text{disp}}$ (3 hidrs fixed, probe hider position varies)

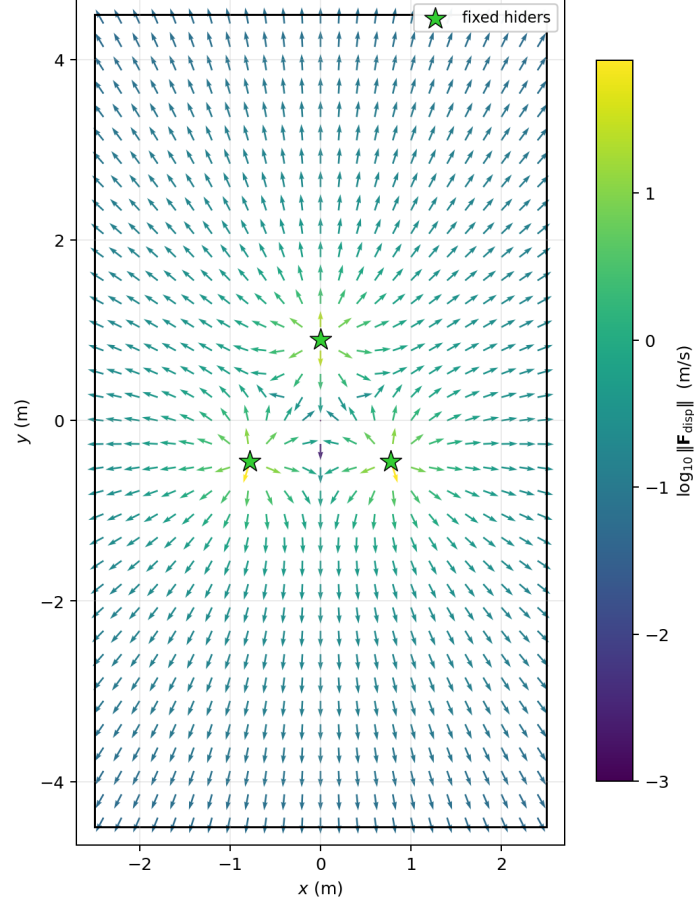


Figure 6: Dispersion field $-\nabla_{\mathbf{p}_i} V_{\text{disp}}$ for a probe hider at every grid point, with three other hidrs held fixed at the marked positions. Arrow direction shows the per-tick velocity command; colour shows its $\log_{10}$ magnitude. The field is radial-out from each fixed neighbour and never goes silent at any distance, which is exactly the property a short-range repulsion would lack.

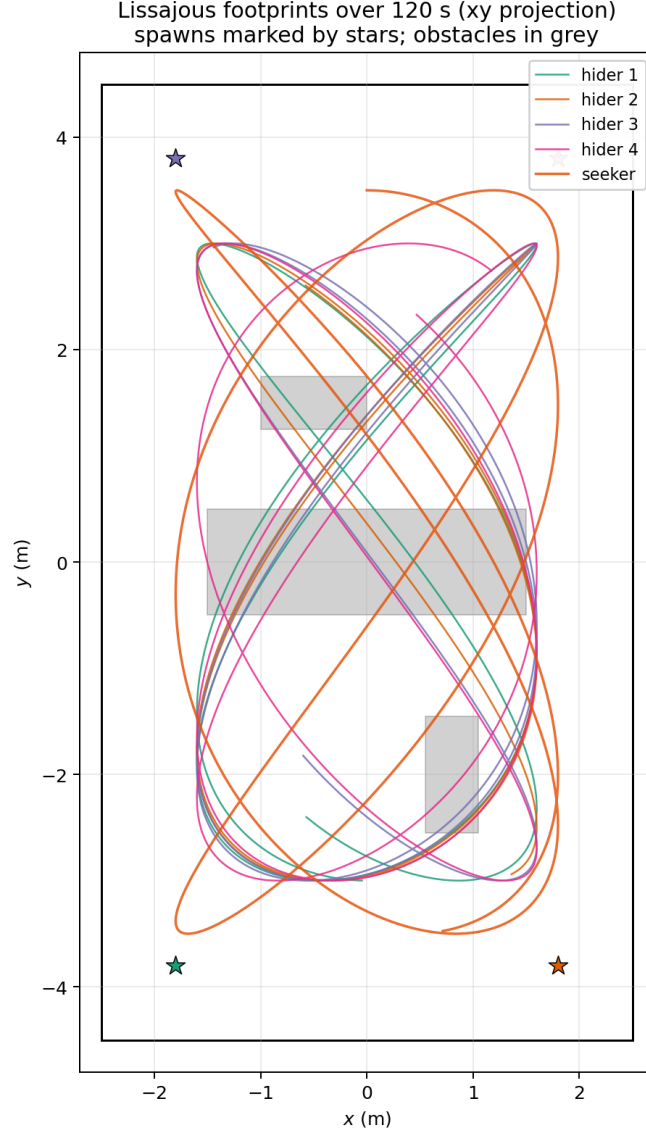


Figure 7: Lissajous footprints over 120 s (xy projection). Each hider’s path uses its individual frequency/phase offsets and the lazy initial-phase offset τ_i described in Sec. 3.2; the spawns (stars) lie on each curve by construction. The seeker’s slow figure-of-eight is drawn in orange. The three obstacle footprints are overlaid in grey – note that the Lissajous orbits cross the obstacle AABs, which is why the obstacle consensus must remain active even when the goal is supposedly on the safe side.

5 Sensing (cf2_sensing.py)

5.1 Intent heading

The heading used both for the FoV cone and the sensing axis is the unit vector toward the current Lissajous target:

$$\hat{\mathbf{h}}(t) = \frac{\mathbf{p}_{\text{lis}}(t) - \mathbf{p}(t)}{\|\mathbf{p}_{\text{lis}}(t) - \mathbf{p}(t)\|}, \quad \text{with } dz := 0 \text{ forced before normalization.}$$

Compared to a path-tangent heading $\dot{\mathbf{p}}_{\text{lis}}/\|\dot{\mathbf{p}}_{\text{lis}}\|$, this points where the drone wants to be rather than where the path is currently moving. The practical consequence: when an obstacle is between the drone and its waypoint, the cone is already aimed at the obstacle. The forced $dz = 0$ (cf2_sensing.py:38) is part of the disabled-Z policy of Sec. 3.2: the heading is a unit vector in the xy plane, and so the rendered cone is a 2D wedge embedded in 3-space.

5.2 FoV-cone obstacle sensing

For each obstacle $\mathcal{B}_j$, the function `sense_obstacles`:

1. Computes the closest box point $\mathbf{c}_j$ to the drone (per-axis clamp).
2. Computes the 3D distance $d_j = \|\mathbf{c}_j - \mathbf{p}\|$ and rejects the obstacle if $d_j > \text{sensing_radius}$.
3. Applies the angular gate $\hat{\mathbf{h}} \cdot \hat{\mathbf{e}}_j \geq \cos \theta_{\text{fov}}$ where $\hat{\mathbf{e}}_j = (\mathbf{c}_j - \mathbf{p})/d_j$, returning the obstacle's $\mathbf{n}_j = -\hat{\mathbf{e}}_j$ and d_j .

A degenerate case is handled explicitly: if $d_j < 10^{-6}$ (drone is on or inside the obstacle) the function flags it as in-cone with $\mathbf{n}_j = -\hat{\mathbf{h}}$, so the downstream consensus law pushes the drone back along the inverse of its heading.

Why hiders sense omnidirectionally. The hider controller passes $\theta_{\text{fov}} = \pi$ to `sense_obstacles`, effectively disabling the angular gate. The forward 45° cone is reserved for *seeker detection*, not obstacle avoidance, since otherwise a hider would be free to enter an obstacle from a blind side. Like the heading, the obstacle-cone test forces $dz = 0$ in the delta to the closest box point (cf2_sensing.py:88); since the cone test is omnidirectional for the hider this is moot in practice, but the same code path is reused (with a real cone) by the seeker – a small bit of asymmetry that should be tightened up if Z is re-enabled.

6 Seeker controller (rmtt_seeker.py)

6.1 Sweep path

The seeker tracks a slow Lissajous sweep:

$$\mathbf{p}_{\text{goal}}^S(t) = \begin{pmatrix} A_X^S \sin(\Omega_X t) \\ A_Y^S \sin(\Omega_Y t + \pi/2) \\ c_z^S + A_Z^S \sin(\Omega_Z t) \end{pmatrix},$$

with $A_X^S = 1.8$, $A_Y^S = 3.5$, $A_Z^S = 0$ (Z disabled, same scaffolding story as the hider in Sec. 3.2), $c_z^S = \text{RMTT_HOVER_Z} = 0.8\text{m}$, and angular frequencies $\Omega_X = 0.18$, $\Omega_Y = 0.13$, $\Omega_Z = 0.08$ rad/s. Ω_Z is kept for symmetry with the x, y branch but is multiplied by $A_Z^S = 0$ so produces no motion. Cycle times are ≈ 35 and 48 s. The $\pi/2$ phase between x and y produces a figure-of-eight footprint that sweeps both “rooms” of the arena every full cycle (overlaid in Fig. 7).

Field-of-view geometry. `can_see` passes iff (i) target in the cone AND (ii) segment apex $\rightarrow$ target hits no AABB.

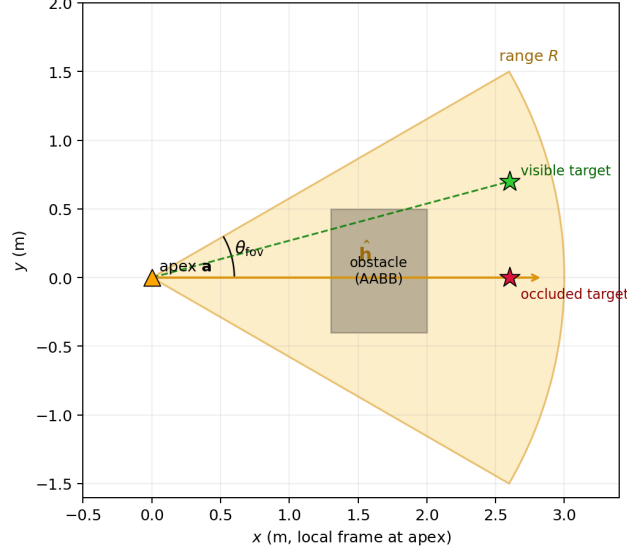


Figure 8: Geometry of `can_see` (`rmitt_seeker.py:153`). The predicate is true iff the target lies inside the cone (range and angular gates both pass) *and* the segment from apex to target intersects no AABB obstacle. The Kay–Kajiya slab algorithm (`_segment_intersects_aabb`) performs the per-axis interval intersection that decides occlusion.

Why the seeker hovers below the hiders. $c_z^S = 0.8\text{m}$ is intentionally below the hiders' 1.0m cruise. Even though the seeker's own heading is computed with $dz = 0$ (`rmitt_seeker.py:71`), the cone test in `can_see` is fully 3D (`rmitt_seeker.py:158`). The 0.2m altitude offset together with the 30° half-angle means a hider directly overhead is already 11° off-axis – not a free catch, but a meaningful geometric constraint that makes the catch easier when the seeker is closer in xy .

6.2 Control law

The seeker uses the same obstacle consensus law as the hiders (with the same parameters $r_{\text{safe}} = 0.6$, $k_{\text{obs}} = 2$, $k_{\text{bar}} = 0.5$) plus a goal attraction $\mathbf{F}_{\text{goal}}^S = k_g^S(\mathbf{p}_{\text{goal}}^S - \mathbf{p})$ with $k_g^S = 0.8$, and the soft boundary repulsion of Sec. 6.3. The xy magnitude is saturated at $v_{\text{max},xy}^S = 0.6\text{m/s}$; the vertical component is clipped to $|v_z^S| \leq 0.6\text{m/s}$ (the same value, so there is no separate vertical cap any more) and then the simulator's `clamp_vel3d` applies the unified 3D actuator clamp at $\text{MAX_V_RMTT} = 0.6\text{m/s}$.

The signature `compute_seeker_cmd(robot_no, robot_pose, obstacle_pose, obstacle_size, clock)` deliberately does *not* accept hider positions: the seeker's *navigation* cannot use any hider state. The `tp_algos.rmmtt_control_fn` wrapper does receive `cf2_poses` (its signature is fixed by the TP framework) and uses them *only* to add the 3D safety-fence repulsion of Sec. 10 – it never forwards them to `compute_seeker_cmd`, so the controller's planning logic remains hider-blind. The same wrapper is what notices when `catch_count() ≥ nbCF2` and sets the seeker's `trigger_land` flag, ending the game; the seeker controller itself has no game-state input.

6.3 Soft boundary repulsion

Unlike the hider, the seeker has no inter-agent dispersion term pulling it inward, only its goal attraction toward the centred Lissajous target. On the slowest axis (Ω_Y) the cycle is 48s and the path swings to $|y| = 3.5\text{m}$ near the wall at $|y| = 4.5$; under unmodelled drift this is close enough that the simulator's 3D clamp can occasionally graze the boundary. The fix (added in

commit `a68365d`) is a soft $1/d$ repulsion of the same form as the hider’s (Sec. 3.2):

$$F_{\text{bnd}}^{S,(w)} = g_{\text{bnd}}^S \left(\frac{1}{\max(d, 10^{-4})} - \frac{1}{m^S} \right), \quad m^S = 0.6, \quad g_{\text{bnd}}^S = 0.3.$$

The gain $g_{\text{bnd}}^S = 0.3$ is $3.3\times$ the hider’s 0.09 – the hider’s wall push is a gentle nudge that the dispersion can easily over-power, while the seeker’s wall push has to do the entire restoring job on its own.

6.4 Search cone and line-of-sight

The search cone has half-angle $\theta_{\text{seek}} = 30^\circ$ and range `SEEKER_VISION_RANGE` = 2.0 m – narrower than the hider’s cone, and *shorter* (earlier experimental builds with a 3.5 m range caught every hider within a single sweep; reducing the range to 2.0 m is what finally made the game winnable for the hidere, because they detect the seeker at 3.0 m and thus have a ~ 1 m head-start to commit to hide mode before they can be tagged). The predicate `can_see(apex, target, heading, fov_half, range, obstacles)` returns `True` when:

1. The 3D distance $\|\mathbf{t} - \mathbf{a}\| \leq \text{range}$.
2. The cone condition holds: $(\mathbf{t} - \mathbf{a}) \cdot \hat{\mathbf{h}} \geq \cos \theta_{\text{fov}} \|\mathbf{t} - \mathbf{a}\|$.
3. No obstacle box intersects the line segment from apex to target.

A concrete catch event captured at simulator level is shown in Fig. 9.

The occlusion test uses the standard Kay–Kajiya slab algorithm (`_segment_intersects_aabb`): for each of the 3 axes, compute the parameter interval $[t_1, t_2]$ over which the segment lies between the slab faces; intersect the three intervals; the segment hits the box iff the intersection overlaps $[0, 1]$. Axis-parallel segments are handled with an explicit “inside the slab?” check.

7 Inter-hider communication (cf2_hider.py)

The shared state is a single module-level dict:

```
1 _last_seen = {'position': None, 'timestamp': None}
2 SEEKER_MEMORY_TTL = 5.0 # seconds
```

Writers (`report_seeker_sighting`) overwrite both fields atomically; readers (`get_last_seen`) compare `t - timestamp` against the TTL and return either the position or `None`. Last-write-wins is fine here because the question is “what is the most recent fix on the seeker”, and between two sightings the older one offers no extra information.

TTL choice. 5 s is enough to keep the swarm in hide mode for several controller cycles after a sighting (the cover-assignment step is itself updated every tick, so the swarm reshuffles quickly inside the window), yet much shorter than a full seeker sweep, so the hidere return to dispersion-driven roaming soon after the threat leaves sight. The exact value is not critical: a shorter TTL biases the swarm toward exploration (more dispersion, less time spent in cover), a longer one biases it toward defence.

Threading. The simulator runs each controller in a `ThreadPoolExecutor`. Python’s GIL plus the atomic-dict-write convention is enough here: a reader can never see a half-written record, only a fully old or fully new one.

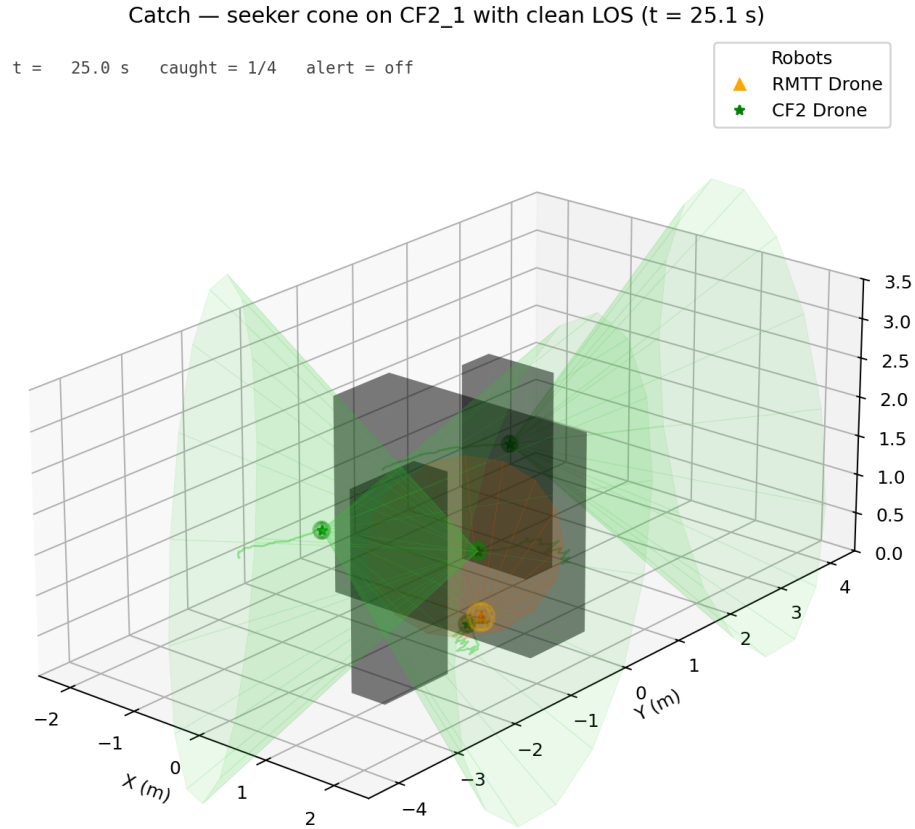


Figure 9: Deterministic catch event. The seeker is positioned 1.4 m south-east of CF2_1 with the cone axis forced toward the hider; the Kay–Kajiya occlusion test passes (no AABB intersects the segment); `game_referee.update_catches` marks the hider on the next tick; and the hider’s controller sees `is_caught(1) = True` and switches to `trigger_land = True` (LED red).

8 Catch detection (game_referee.py)

The referee owns a `set` of caught hider IDs and a dict of catch times. Each frame it iterates over (flying seekers) $\times$ (flying, not-yet-caught hiders), calls `can_see` (the same predicate the seeker would compute internally if it had one), and adds the hider to the caught set if visible. Once caught, a hider stays caught for the rest of the simulation. The hider controller in `tp_algos.cf2_control_fn` reads this flag via `is_caught(robot_no)` and returns `trigger_land = True`, which puts the drone into landing state.

End-of-game: the seeker lands too. The seeker wrapper in `tp_algos.rmtt_control_fn` polls `catch_count()` every tick; when it reaches `nbCF2` (all hiders caught), it short-circuits the call to `compute_seeker_cmd`, emits `(0,0,0)` as the velocity command, and sets `trigger_land = True` (lines 275–279). The simulator then drives the seeker through its state-3 landing transient (linear descent over `LANDING_TIME = 3s`, same as the takeoff except in reverse) and returns it to idle. The referee’s `_caught` set persists for the module’s lifetime, so re-takeoff in the same Python process replays a frozen game state – a minor wart that is flagged in Future Work.

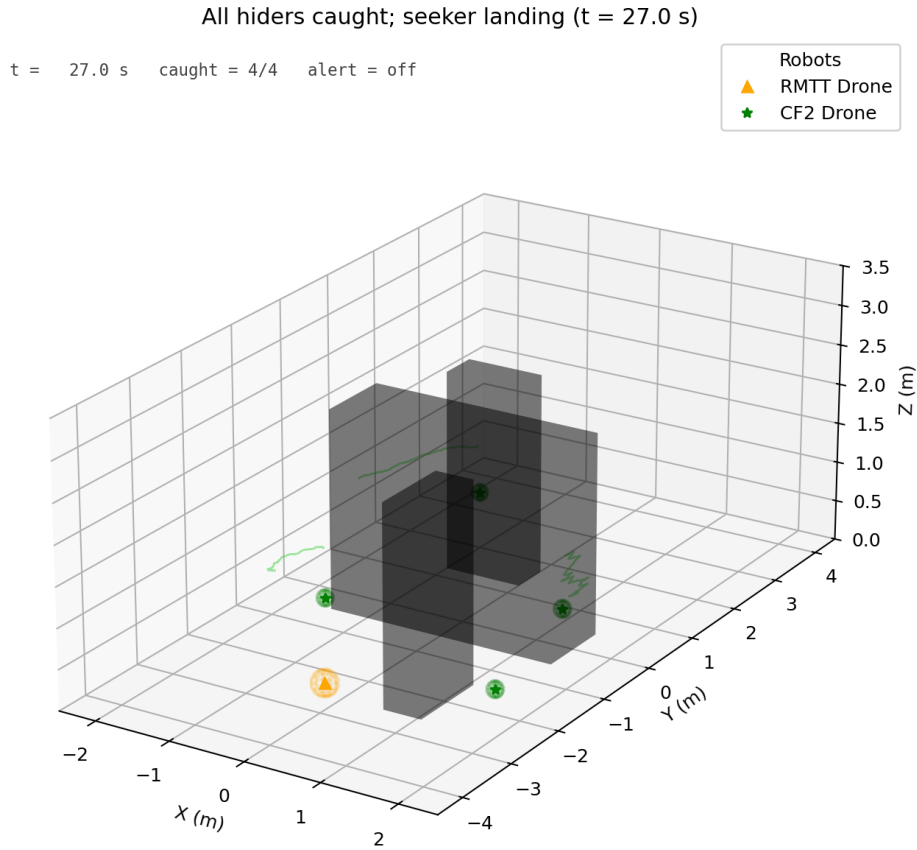


Figure 10: End of game: `catch_count() = 4/4`. The four hiders have triggered `trigger_land` and are descending or already on the floor; the seeker has entered its own state-3 landing transient and is cruising downward in place. The HUD shows the final state.

9 Visualization (sim_viz.py)

The visualization layer adds three things on top of the simulator’s basic markers:

- **3D FoV cones.** For each flying drone, a triangle-fan polyhedron is built with apex at the drone and axis along its heading. The 16-segment base disk is constructed in the plane perpendicular to the heading using an orthonormal basis built from an arbitrary reference vector (world- z , switching to world- x when the heading is nearly vertical). The heading the cone uses is `hider_view_heading` for hiders and `seeker_heading` for the seeker; in the hider case this is the *same* function the controller passes to `can_see`, so the on-screen cone and the cone that actually triggers a sighting are guaranteed to coincide (see Sec. 3.4, “One heading function for both vision and rendering”).
- **Position trails.** Each hider keeps a bounded deque (`TRAIL_LEN = 80`) of recent positions, drawn as a thin green line. When the hider returns to idle (after a catch landing) the trail is cleared so the next takeoff starts fresh.
- **HUD.** A small monospace text in the upper left shows t , the catch count, and an **alert on/off** flag derived from `get_last_seen(t)`.

The cone colour for the hiders switches from **limegreen** (roam) to **gold** (alert) when a fresh sighting is in shared memory, so the global mode is visible at a glance.

10 Simulator hooks and controller-level separation (tp_algos.py)

The TP framework requires every controller to live in `tp_algos.py` with a fixed signature. Per the project rules, only the body marked **TO BE MODIFIED** may be edited. Two responsibilities live here:

1. Dispatch each robot type to its dedicated module (`cf2_milestone1`, `rmtt_seeker`, ...).
2. Apply a controller-level 3D drone-to-drone separation *on top of* what each module returns. This is the only place in the code base that has simultaneous access to *both* hider and seeker poses, so it is the natural place to enforce a global “no two drones get closer than 1 m” rule.

10.1 3D inter-drone barrier: a hard 1 m floor

`_compute_drone_repulsion_3d(self_pose, neighbors, min_dist=1.0, activation=1.5)` enforces a *hard* minimum separation $d_{\min} = 1\text{ m}$ between any pair of drones (hider-hider and hider-seeker alike). For each neighbor k with $d_{ik} < d_{\text{act}}$:

$$\Delta \mathbf{v}_i = g_{\text{rep}} \sum_{k: d_{ik} < d_{\text{act}}} \left(\frac{1}{d_{ik} - d_{\min}} - \frac{1}{d_{\text{act}} - d_{\min}} \right) \frac{\mathbf{p}_i - \mathbf{p}_k}{\|\mathbf{p}_i - \mathbf{p}_k\|}, \quad g_{\text{rep}} = 30, \quad d_{\text{act}} = 1.5\text{ m}.$$

The magnitude vanishes continuously at $d = d_{\text{act}}$ and diverges as $d \rightarrow d_{\min}^+$. If a pathological state $d \leq d_{\min}$ is ever observed (e.g. at spawn), the function returns a 10^6 -magnitude outward command which, after the simulator’s per-axis clamp at $\text{MAX_V} = 0.6\text{ m/s}$, drives the drone at full speed purely outward.

Discrete-time non-penetration argument. Let $\Delta t = 0.05\text{ s}$ and $v_{\max} = 0.6\text{ m/s}$. The largest closure two drones can produce in one tick is $2v_{\max}\Delta t = 0.06\text{ m}$. Suppose at tick k we have $d_k > d_{\min}$.

- If $d_k \geq d_{\text{act}}$, the barrier is zero on this step but $d_{k+1} \geq d_k - 0.06 \geq d_{\text{act}} - 0.06 = 1.44\text{ m} > d_{\min}$.

- If $d_{\min} < d_k < d_{\text{act}}$, the barrier magnitude already exceeds the controllers' bounded xy commands (≤ 0.45 m/s for the hider, ≤ 0.55 m/s for the seeker) once $d_k \leq d_{\text{act}} - 0.06$. The clamp therefore resolves the net velocity to $v_{\max}$ purely outward on both drones, and $d_{k+1} \geq d_k + 2v_{\max}\Delta t > d_{\min}$.

Combining the two cases, $d_k > d_{\min} \Rightarrow d_{k+1} > d_{\min}$ for every k . The 1 m floor is invariant under the closed-loop dynamics.

A numerical degenerate case ($d_{ik} < \varepsilon = 10^{-6}$ m) returns a fixed $+x$ push so the function never divides by zero. The sum operates in all three axes, contrary to the planar dispersion consensus in `cf2_milestone1` which only sees xy ; the 3D component matters when a hider and the seeker happen to be vertically aligned (which the dispersion law ignores by construction). The two laws compose by addition, so the barrier acts as a short-range 3D hard fence on top of whatever long-range xy geometry the dispersion law has chosen.

10.2 How the helper is used

- `cf2_control_fn`: once the hider is flying and is not caught, it calls `compute_roaming_cmd` for $(v_x, v_y, z_{\text{dist}}, \text{led})$. It then collects *all other* flying CF2s *and all* RMTT seekers as neighbors, asks `_compute_drone_repulsion_3d` for $(\Delta v_x, \Delta v_y, \Delta v_z)$, and adds the result to the controller's (v_x, v_y) and to a synthesized $v_z = z_{\text{dist}} - z$. The corrected v_z is then folded back into z_{dist} so the simulator's " $v_z = z_{\text{dist}} - z$ " convention is preserved.
- `rmtt_control_fn`: gets $(v_x, v_y, v_z, \text{led})$ from `compute_seeker_cmd` (which itself never sees hider poses), then collects all flying CF2s as neighbors and adds the 3D repulsion to (v_x, v_y, v_z) . This is the one place where the seeker controller layer actually reads `cf2_poses` – but only to keep its distance, not to chase. The asymmetry of the game (the seeker has to *see* the hiders to catch them) is still preserved because the catch logic in `game_referee` uses vision alone.
- `tb3B_control_fn`, `tb3W_control_fn`, `rmep_control_fn`: placeholder goal-seek behaviour $(v_x, v_y = 0.2(g - p)$ toward a hard-coded g). These robots are not part of the current hide-and-seek configuration (`nbTb3B = nbTb3W = nbRMEP = 0`).

10.3 Hider takeoff handshake

The CF2 takeoff is split between the simulator's state machine and a small per-controller cache in `cf2_control_fn`. The cache (`_takeoff_done_by_robot`, indexed by `robot_no`) starts at `False`; while the drone sits at $z < 0.05$ m it returns `trigger_takeoff = True` and shows a blue LED. The simulator then runs the 3 s linear takeoff (state 1) up to `CF2_HOVER_Z`; once the controller sees $z > 0.1$ m it flips the cache to `True` and from then on delegates to `compute_roaming_cmd`. If the referee marks the hider caught, the wrapper sends `trigger_land = True` and lights the LED red.

The lazy-import trick (`if not hasattr(..., "_cached"): import ...`) keeps the module fast to reload and avoids polluting the framework's expected namespace.

11 Simulation infrastructure (`standalone_sim.py`)

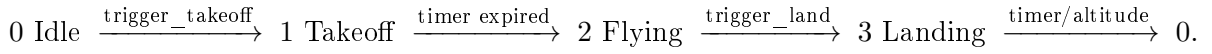
11.1 World and hardware specs

Constant	Value	Meaning
<code>dt</code>	0.05 s	Simulation step.
<code>X_MIN</code> , <code>X_MAX</code>	± 2.5 m	Arena bounds (mirrored in <code>cf2_milestone1</code>).
<code>Y_MIN</code> , <code>Y_MAX</code>	± 4.5 m	

Z_MIN, Z_MAX	0, 3.5 m	
MAX_V_CF2	0.6 m/s	3D speed cap for hider drones.
MAX_V_RMTT	0.6 m/s	3D speed cap for the seeker.
RAD_CF2	0.05 m	CF2 safety glob is 2× this.
RAD_RMTT	0.08 m	RMTT safety glob is 2× this.
CF2_HOVER_Z	1.0 m	Target hover altitude after takeoff.
RMTT_HOVER_Z	0.8 m	
TAKEOFF_TIME	3.0 s	Linear takeoff over [0, HOVER_Z].
LANDING_TIME	3.0 s	
DRONE_POS_NOISE_STD	0.005 m	Per-step Gaussian noise on each axis.

11.2 State machine

Each drone has a 4-state machine:



In states 1 and 3 the altitude is driven linearly by the takeoff/landing timer; in state 2 the controller’s (v_x, v_y, v_z) are applied with the actuator saturation and noise step.

11.3 Async controller execution

The simulator runs each controller in a `ThreadPoolExecutor` with `max_workers = 20`. The helper `get_async_cmd` submits a job if none is running for the given robot, and otherwise returns the cached last command. This means a controller can block (e.g. `time.sleep`) without stalling the simulation – the drone simply keeps its last command. The trade-off is that controllers can be running on slightly stale snapshots, but in practice the world barely changes in one `dt`.

11.4 Collision detection

Each robot has a “safety glob” of radius $2 \times$ base that turns red on contact. Three checks per frame:

- **Boundary:** arena walls in x, y ; ceiling at $z = Z_{\max}$ always; floor at $z = 0$ only for drones in the flying state.
- **Obstacle:** closest-point distance against each AABB (same per-axis-clamp formula as the sensing module).
- **Robot-to-robot:** 3D distance against the sum of the two globs’ radii.

Warnings are rate-limited to one per (robot, kind) per second.

12 Tunables summary

Name	Value	Where / why
r_{safe}	0.6 m	Desired clearance from obstacles (<code>R_SAFE</code> in both hider and seeker).
k_{obs}	2 (1/s)	Linear consensus gain.
k_{bar}	0.5 m/s	Barrier gain near contact.
ε	$0.02 r_{\text{safe}}$	Numerical clamp inside the barrier.

$\theta_{\text{fov}} \text{ hid}$	45°	Forward cone for seeker detection and visualization. Reduced from 60° to suppress false alerts.
sensing radius	2.0 m	Obstacle sensing range (hider and seeker).
seeker detection range	3.0 m	Hider-side range to detect the seeker.
k_{disp}	0.4	Inter-agent dispersion gain (cf2_milestone1.K_DISP); scale of the Coulomb-form swarm push.
$\varepsilon_{\text{disp}}$	0.05 m	Pairwise-distance floor inside the dispersion law (numerical safety).
k_g^{roam}	0.45	Hider goal attraction in roam mode, used together with the analytic Lissajous feed-forward $\dot{\mathbf{P}}_{\text{lis}}$.
k_g^{hide}	0.9	Hider goal attraction in hide mode (cover target is static, no feed-forward).
d_{min}	1.0 m	3D drone-to-drone hard floor; invariant under the closed-loop dynamics (tp_algos.DRONE_MIN_DISTANCE).
d_{act}	1.5 m	Activation buffer above d_{min} where the barrier turns on; sized so $d_{\text{act}} - d_{\text{min}} \gg 2v_{\text{max}}\Delta t = 0.06$ m (tp_algos.DRONE_ACTIVATION_DISTANCE).
g_{rep}	30.0	3D drone-to-drone barrier gain (tp_algos.DRONE_REPULSION_GAIN).
hider boundary margin	0.6 m	Distance at which the hider's wall push turns on.
$g_{\text{bnd}} \text{ hider}$	0.09	Hider wall push gain.
seeker boundary margin	0.6 m	Distance at which the seeker's wall push turns on.
$g_{\text{bnd}}^S \text{ seeker}$	0.3	Seeker wall push gain (3.3× the hider's; the seeker has no dispersion to pull it inward).
$v_{\text{max}}^{\text{xy}} \text{ hider}$	0.55 m/s	Hider final xy saturation.
$v_{\text{max}}^{\text{xy}} \text{ seeker}$	0.6 m/s	Seeker final xy saturation (= MAX_V_RMTT; the seeker uses its whole speed budget).
$v_{\text{max}}^z \text{ seeker}$	0.6 m/s	Seeker z saturation (same as xy – no separate cap in the current build).
k_g^S	0.8	Seeker goal attraction gain.
seeker vision range	2.0 m	Seeker FoV slant length.
Z rate limit hider	0.01 m/call	Per-call $ z_{\text{dist}} - z $ cap, protects the xy saturation against the simulator's unified 3D actuator clamp.
d_{cov}	0.8 m	Cover offset past the obstacle surface.
TTL	5 s	Lifetime of a seeker sighting in shared memory.
θ_{seek}	30°	Seeker FoV half-angle.
$\Omega_X^S, \Omega_Y^S, \Omega_Z^S$	0.18, 0.13, 0.08 rad/s	Seeker Lissajous.
A_X^S, A_Y^S, A_Z^S	1.8, 3.5, 0.0 m	Seeker Lissajous; $A_Z^S = 0$ is the disabled-Z policy.
$\omega_x^{(i)}$	0.16 + 0.018($i - 1$) rad/s	Hider Lissajous (per-drone, x).
$\omega_y^{(i)}$	0.11 + 0.015($i - 1$) rad/s	Hider Lissajous (per-drone, y).

A_x, A_y	1.6, 3.0 m	Hider Lissajous amplitudes in xy .
A_z	0.5 m	Scaffolding only – commented out in both the position target and the FoV axis.
τ_i	coarse-searched	Per-drone Lissajous phase offset, picked so $\mathbf{p}_{\text{lis}}^{(i)}(\tau_i)$ is nearest to spawn (kills the spawn lurch).
<code>_COVER_CACHE_MAX_AGE</code>	1.0 s	Cover-assignment cache TTL.
<code>_COVER_CACHE_SEEKER_DELTA</code>	0.5 m	Cover-assignment seeker-motion gate (re-solve if the seeker has moved more than this since the cache was built).
<code>TRAIL_LEN</code>	80 frames	Trail length in the visualization.

13 Discussion and design choices

Why separate the FoV cone from the obstacle sensor. The first iteration of the hider used the same 60° cone for both seeker detection and obstacle avoidance. The result: the drone would happily back into a wall when its heading was pointing forward. Splitting the two (omnidirectional for obstacles, forward cone for vision) is what makes the hider safe in a cluttered arena while still “looking somewhere”.

Why a barrier on top of the linear consensus. A pure linear law with the same k_{obs} would let the goal-attraction push the drone arbitrarily close to an obstacle if the goal is on the far side. The barrier term grows like r_{safe}/d , so it eventually dominates any *bounded* attraction; combined with the magnitude saturation at the actuator limit, this gives “always-bounded velocity, never collides” under reasonable goals.

Why cover behind the surface, not the center. For elongated obstacles, a cover point at distance r_{cov} from the obstacle center can still be inside the AABB. The implementation computes the parametric ray-AABB exit and adds the offset *past* the surface; combined with $d_{\text{cov}} > r_{\text{safe}}$ this gives a cover point that sits in the safe set, so the hider can rest there without being pushed away.

Why a gradient flow (Coulomb potential) for inter-hider dispersion. The earlier implementation used a short-range inverse-square push with a 0.9 m cutoff – effective for collision avoidance but invisible beyond that radius, so the swarm could perfectly well sit on four parallel Lissajous orbits and offer the seeker a tightly-correlated target. Replacing it with $\mathbf{u}_i = -\nabla_{\mathbf{p}_i} V_{\text{disp}}$ on the Coulomb potential gives (i) a single aggregate cost that all agents collectively minimize – the textbook multi-agent consensus signature (ii) a force that never goes silent, so any contraction of the swarm is met by an immediate restoring push, and (iii) a closed-form Lyapunov certificate. The price is the equilibrium configuration depends on the boundary repulsion, which is why g_{bnd} stays small (0.09): strong walls would push the swarm into the centre and undo the dispersion.

Why an explicit cover assignment, not nearest-cover-per-agent. With four hidere and two obstacles, the “each hider runs to its closest cover” policy frequently sends two hidere to the same face. Once they arrive, the dispersion push spreads them but they remain on the same *visible* side of the obstacle – a single seeker sweep is enough to catch both. Solving a min-cost matching makes the assignment a global property of the swarm, and the candidate set is built per-face (not per-obstacle) so the granularity matches the number of hidere. Decentralized execution

falls out of determinism: every hider computes the same optimization on the same inputs and reads its own row.

Why hide the hider positions from the seeker controller specifically. The *planning* module of the seeker, `compute_seeker_cmd`, does not accept `cf2_poses`. The `tp_algos` wrapper still has access to them (its signature is dictated by the framework) and uses them only for the 3D safety-fence repulsion. The split is deliberate: a safety fence is a property of the world, not of the game (we do not want drones colliding mid-air); whereas the search behaviour must remain vision-only, which is what the referee enforces via `can_see`. Future changes to the seeker’s search logic would have to explicitly thread hider positions through `compute_seeker_cmd`’s signature, which would be visible in review.

Why shared-memory comms rather than message passing. The simulator already runs each controller in a thread; a single last-write-wins dict is the smallest faithful model of “every hider can hear every other hider’s report”. A more elaborate scheme (per-pair channels, ack/nak, etc.) would not change the closed-loop behaviour because the controllers only care about “most recent fix on the seeker”.

Why the seeker can outrun the hidere raw, yet still loses. The xy speed caps are $v_{\max}^{\text{hider}} = 0.55 \text{ m/s}$ and $v_{\max}^{\text{seeker}} = 0.6 \text{ m/s}$ – the seeker is marginally faster. The hidere still win because of two range asymmetries: their detection range is 3.0 m (against the seeker’s 2.0 m vision range), and their reaction is collective (a single sighting commits the whole swarm to hide mode via shared memory). Before the seeker is close enough to catch any one hider, every hider has already seen it and is $\sim 0.55 \text{ m/s} \times \tau_{\text{react}}$ on its way to cover. τ_{react} is just the single tick of latency between the sighting being written and `get_last_seen` being read.

Why the cover-assignment cache, not pure instantaneous matching. The min-cost matching is deterministic in the seeker’s last-seen position; on near-ties between two candidate assignments, a tiny perturbation ($\sim 5 \text{ mm}$ of position noise) is enough to flip which hider takes which face. Without hysteresis, two hidere would swap roles every few frames, each adding a wasted round-trip to its trajectory. The $1 \text{ s} / 0.5 \text{ m}$ cache gate kills the swap chatter without changing the steady-state assignment (both candidates were optimal up to the tie, so either choice is fine). As a side effect, the brute-force matching runs at $\sim 1 \text{ Hz}$ instead of 20 Hz – a $20\times$ saving that would matter on real hardware.

14 Future work

- **Multiple seekers and adversarial cooperation.** The `can_see` predicate is already multi-seeker aware (the referee loops over all flying RMTTs). What is missing is a seeker coordination law that splits the arena and avoids redundant sweeps.
- **Bounded consensus with a soft centroid anchor.** The inter-agent dispersion law is purely repulsive; bounded equilibria currently come from the arena walls. Adding a weak attractive term toward a moving centroid (or toward an assigned region centroid in a Voronoi tessellation) would give a true bounded-consensus behaviour and would let the swarm hold formation away from the walls.
- **Re-enable Z.** The scaffolding is in place: $A_z = 0.5$, $\omega_z^{(i)} = 0.09 + 0.013(i - 1)$, $\varphi_z^{(i)}$, and the z branch of `intent_heading/sense_obstacles`. Each is currently commented out (Sec. 3.2). Re-enabling Z requires (i) giving v_z its own private saturation cap so the xy avoidance budget

is not eaten by the 3D actuator clamp under a Z excursion, (ii) re-validating the obstacle-barrier discrete-time argument in 3D, and (iii) extending the Coulomb dispersion to a full 3D gradient sum. The dispersion law promotion is trivial; the actuator and barrier checks are the load-bearing work.

- **Persistent referee state across runs.** The catch set and the shared sighting persist for the lifetime of the Python process, so a re-takeoff inside the same simulator session inherits a frozen game state. A `game_referee.reset()` call (called either by the simulator on the next state-0 transition or by the user explicitly) would make the sim hot-reloadable for demo use.
- **Connectivity-preserving dispersion.** The current Coulomb law diverges at zero distance and decays at infinity, so the swarm equilibrium pushes against the boundary. A connectivity-preserving alternative (e.g. a potential with a finite well at a target spacing) would let the swarm stay inside a sensing radius for explicit position broadcasts, matching the wireless-range constraint of the real Voliere hardware.

A File listing

File	LoC (approx.)	Purpose
<code>cf2_milestone1.py</code>	~ 250	Main hider controller; also exports <code>hider_view_heading</code> and <code>_t_offsets</code> .
<code>cf2_sensing.py</code>	~ 120	FoV sensing primitives.
<code>cf2_consensus.py</code>	~ 70	Obstacle and Coulomb consensus laws.
<code>cf2_hider.py</code>	~ 260	Inter-hider comms + cover geometry + cover-assignment cache.
<code>rmitt_seeker.py</code>	~ 180	Seeker controller (+ soft boundary repulsion) + <code>can_see</code> .
<code>game_referee.py</code>	~ 50	Catch detection.
<code>sim_viz.py</code>	~ 160	Cones, trails, HUD.
<code>tp_algos.py</code>	~ 440	Required entry-point shim plus controller-level 3D drone-to-drone repulsion.
<code>standalone_sim.py</code>	~ 420	The simulator (4 hidrs, 3 obstacles in the default spawn config).
<code>scripts/render_scenes.py</code>	~ 240	Headless screenshot driver: drives <code>standalone_sim.update()</code> manually, with a per-scene <code>reset_world</code> , catch and cone toggles, and a deterministic catch via a forced <code>seeker_heading</code> .
<code>scripts/render_law_plots.py</code>	~ 230	Analytical figures (barrier curves, dispersion field, Lissajous footprint, FoV schematic).
<code>report/figures/*.png</code>	10 figures	Figures used throughout the report; regenerable by re-running the two scripts above.